Werner-von-Siemens-Schule Mannheim

„Interaktives ‚Wer wird Millionär'-Quiz mit ESP32-S3, TFT-Touchscreen und SD-Karte"

Antonio Okwarauba

Klasse: 2BKI21

Datum: 28. April 2026

Projektbetreuer: Herr Spitzer

Inhaltsverzeichnis

1 Einleitung....3
2 Planung....3
2.1 Hardwareplanung....3
2.2 Softwareplanung....4
2.3 Ablaufplanung....4
2.3.1 GanttProject....4
2.3.2 Zeitplan....4
2.4 Aufbau der Fragen / Datenorganisiation....5
2.5 Test- und Validierungsplanung....5
3 Grundlagen....5
3.1 Hardware-Grundlagen....6
3.2 Software-Grundlagen....6
3.3 Zusammenfassung....7
4 Realisierung....7
4.1 Hardware-Realisierung....8
4.1.1 Verdrahtungsplan des ESP32-S3 mit TFT Display und SD-Karte....8
4.1.2 Pinbelegungstabelle....8
4.2 Realisierung der Testfunktionen....9
4.3 Realisierung der Gesamtfunktion....10
4.4 Benutzeroberfläche und Spielablauf....11
4.4.1 Menübildschirm....12
4.4.2 Spielbeginn-Bildschirm....13
4.4.3 Frage- und Antwortbildschirm....14
4.4.4 Game-Over-Bildschirm....16
4.5 Zustandsautomat....17
4.6 Datenverarbeitung mit Sd-Karte und JSON....18
4.7 Umsetzung der Text- und Antwortdarstellung....20
5 Fehlersuche, Beisetungen und Verbesserung....21
5.1 Fehlersuche und Fehlerursachen....22
5.1.1 Fehler bei der SPI-Verkabelung des TFT-Displays....22
5.1.2 Fehler bei der SD-Karten-Verkabelung....22
5.1.3 SPI-Bus- und Ressourcen-Konflikt beim ESP32-S3....23
5.2 Fehlerbeseitungen....23
5.2.1 Fehlerbehebung – TFT- Display(Reboot und weißer Bildschirm)....23
5.2.2 Fehlerbehebung – SD-Karte (Initialisierung und SD“fail“)....23
5.2.3 Fehlerbeseitung zum SPI-Bus und Ressourcen Konflikt beim ESP32S3....24
5.3 Erkenntnisse und Verbesserungen....24
6 Ergebnisse....25
6.1 Gehäuseentwurf....25
7 Fazit....28
8 Anhang und Quellen....28
8.1 Programmcode....28
8.2 Nutzung von KI....29
8.2.1 Prompt für die Software Struktur....29
8.2.2 Prompt für den Software-Ablauf....29
8.2.3 Prompt-Optimierung für Umlaute und Zeilenumbrüche....29
8.3 Verwendete Bibliotheken....30
8.4 Web-Quellen....30
9 Bedienungsanleitung....31
10 English Summary....31

1 Einleitung

In diesem Projekt wird ein interaktives Quizspiel auf Basis eines ESP32-Mikrocontrollers und eines 3,5-Zoll-TFT-Displays (ILI9488) realisiert. Die Spiel-Logik orientiert sich am bekannten „Wer wird Millionär“, die Inhalte bestehen jedoch aus Fragen zur Digitaltechnik, wodurch das Projekt sowohl spielerisch als auch lehrreich ist.

Ziel des Projekts

Das Ziel dieses Projekts ist es, ein funktionierendes Lern-Quiz zu Digitaltechnik zu entwickeln, das auf einem ESP32-Mikrocontroller mit Touchdisplay läuft. Das System soll Fragen stellen, Antworten auswerten und den Spielablauf korrekt steuern. Gleichzeitig soll es praxisnahe Erfahrungen in Mikrocontroller-Programmierung, Touchscreen-Bedienung und Display-Ansteuerung vermitteln.

Warum ich dieses Thema gewählt habe

Ich habe dieses Thema gewählt, weil ich die Herausforderung interessant finde, ein bekanntes Spielprinzip digital umzusetzen und dabei praxisnah Erfahrungen in Mikrocontroller-Programmierung, Touchbedienung und Display-Ansteuerung zu sammeln. Durch die Anpassung der Fragen auf digitale Technik kann das Projekt zudem Wissen auf anschauliche Weise vermitteln.

Ausblick auf die Planung

Nach der Einführung in das Projekt werden im nächsten Abschnitt die Planung und Vorbereitung beschrieben. Dazu gehört die Auswahl der Hardware-Komponenten wie ESP32 und TFT-Touchscreen, die Auswahl der Softwarebibliotheken für die Display- und Touch-Steuerung sowie die Strukturierung der Quizlogik

2 Planung

Um das in der Einleitung definierte Ziel – ein interaktives Lern-Quiz zu Digitaltechnik auf ESP32 mit TFT-Touchdisplay zu erreichen, werden im Folgenden die Hardware- und Softwareauswahl, der Ablauf, die Frageorganisation sowie der Zeitplan beschrieben.

2.1 Hardwareplanung

Für die Umsetzung werden folgende Hardware-Komponenten eingesetzt:

- **Mikrocontroller:** ESP32-S3, zuständig für die Steuerung und Logik des Spiels.
- **Display mit Touchfunktion:c**3,5-Zoll-TFT ILI9488, zur Darstellung der Fragen und Antworten sowie zur Eingabe der Antworten per Berührung
- **SD-Karte:** enthält die Fragen und die richtigen Antworten, die das Programm beim Quiz abruft, wodurch Inhalte flexibel geändert oder erweitert werden können.

- **Verbindungen:** Jumperkabel für alle Verbindungen zwischen ESP32, Display und SD-Karte.
- **Stromversorgung:**Powerbank, um das System kabellos betreiben zu können

2.2 Softwareplanung

Die Software wird in der **Arduino IDE** entwickelt. Folgende Bibliotheken kommen zum Einsatz:

- **TFT_eSPI:** für die Ansteuerung des Displays, Darstellung von Text und Grafiken sowie für die Touch-Funktion.
- **XPT2046_Touchscreen:** für die Toucherkennung auf dem Display (Auswertung von Berührungen, z. B. Auswahl der Antworten).
- **SD:** zum Zugriff auf die SD-Karte, auf der die Fragen gespeichert sind.
- **ArduinoJson:** zum Einlesen der Fragen aus der JSON-Datei auf der SD-Karte. Die Bibliothek ermöglicht das strukturierte Verarbeiten der gespeicherten Daten im Programm.

Die Softwarestruktur ist modular aufgebaut:

- **Quizlogik:** Steuerung von Fragen, Antworten, Punkten und Spielablauf.
- **Anzeige:** Darstellung von Fragen, Antworten und evtl. Punkteständen
- **Touchabfrage:** Prüft Berührungen auf dem Display und wertet sie als Antwort aus, über die XPT2046-Bibliothek.
- **Datenorganisation:** Fragen und die zugehörigen richtigen Antworten werden von der SD-Karte eingelesen. Dadurch können Inhalte flexibel geändert oder erweitert werden, ohne die Softwarestruktur anzupassen.

2.3 Ablaufplanung

2.3.1 GanttProject

GanttProject wurde zur Planung und Organisation des Projekts eingesetzt. Mithilfe eines Gantt-Diagramms konnten die einzelnen Arbeitsschritte strukturiert geplant und zeitlich eingeordnet werden. Dadurch war es möglich, den Fortschritt des Projekts übersichtlich zu verfolgen und den Zeitplan einzuhalten.

Da das vollständige Gantt-Diagramm aufgrund seiner Größe nicht übersichtlich in die Dokumentation integriert werden konnte, wurde stattdessen eine tabellarische Darstellung des Zeitplans erstellt.

2.3.2 Zeitplan

Phase	Aufgabe	Zeitraum
1	Hardware Auswahl und Bestellung	20.10.25 – 02.11.25
2	Hardwareaufbau und Verbindungstest + touch	03.11.25 – 16.11.25
3	Grundstruktur des Quizprogramms	17.11.25 – 30.11.25
4	Spiellogik implementieren	01.12.25 – 14.12.25
5	Benutzeroberfläche gestalten	15.12.25 – 21.12.25
6	Touch eingabe mit Spiellogik verknüpfen	05.01.26 – 18.01.26
7	Projekt test, Fehlerbehebung, (evtl. Erweiterungen)	19.01.26 – 25.01.26

2.4 Aufbau der Fragen / Datenorganisiation

Die Fragen zur Digitaltechnik werden auf der SD-Karte gespeichert. Das System ist so gestaltet, dass Inhalte jederzeit geändert oder erweitert werden können, ohne die grundlegende Softwarestruktur zu verändern.

2.5 Test- und Validierungsplanung

Es werden sowohl Einzelfunktionen als auch das Gesamtsystem getestet:

- Überprüfung der Touch-Reaktionen auf dem Display
- Korrekte Darstellung der Fragen und Antworten
- Zugriff auf die SD-Karte und Einlesen der Fragen
- Richtiges Spielablauf evtl. auch richtige Punktevergabe
- Gesamtintegration: Überprüfung, ob das Spiel stabil und fehlerfrei läuft

Durch diese Planung wird sichergestellt, dass das Projekt systematisch, nachvollziehbar und effizient umgesetzt werden kann.

3 Grundlagen

In diesem Kapitel werden die für das Projekt relevanten Bauteile und Bibliotheken beschrieben. Ziel ist es, die eingesetzten Komponenten, ihre Funktionen und die verwendeten Softwarebibliotheken zu erklären, um das Verständnis für die spätere Umsetzung zu schaffen.

3.1 Hardware-Grundlagen

ESP32-S3 Mikrocontroller

Der ESP32-S3 ist ein leistungsfähiger Mikrocontroller mit integriertem WLAN und Bluetooth, optimiert für KI- und Touch-Anwendungen. Er wird für die Steuerung des Quizsystems eingesetzt, verarbeitet Eingaben vom Touchdisplay, liest Daten von der SD-Karte aus und steuert die Anzeige sowie Punktevergabe.

3,5-Zoll-TFT-Display mit Touchfunktion (ILI9488)

Dieses Display zeigt die Fragen, Antworten und Punktestände an. Die integrierte Touchfunktion ermöglicht die direkte Eingabe durch Berührung, sodass Spieler die Antworten auswählen können.

SD-Karte

Auf der SD-Karte werden die Quizfragen und die richtigen Antworten im **JSON-Format** in einer json Datei gespeichert. Dadurch kann das Programm die Daten strukturiert einlesen, leicht auswerten und die Inhalte jederzeit angepasst oder erweitert werden, ohne die Software selbst zu ändern. Die SD-Karte wird über SPI mit dem ESP32-S3 verbunden.

Jumperkabel

Jumperkabel dienen zur Verbindung der einzelnen Bauteile auf dem Breadboard sowie vom Mikrocontroller zu anderen Geräte. Sie stellen sicher, dass Signale und Spannungen korrekt übertragen werden.

Powerbank

Ermöglicht den kabellosen Betrieb des Systems, sodass das Quiz mobil und kabellos eingesetzt werden kann.

3.2 Software-Grundlagen

Arduino IDE

Die Arduino IDE wird zur Programmierung des ESP32-S3 genutzt. Sie bietet eine benutzerfreundliche Umgebung zur Erstellung, Kompilierung und Hochladung des Codes.

SPI Bibliothek (SPI.h)

Die SPI-Bibliothek stellt die serielle Schnittstelle **SPI (Serial Peripheral Interface)** zur Verfügung. Sie ermöglicht die Kommunikation zwischen dem ESP32-S3 und externen Komponenten wie dem TFT-Display, dem Touchcontroller und der SD-Karte. Die Bibliothek regelt den Datentransfer über die Leitungen MOSI, MISO, SCK und CS und bildet damit die Grundlage für die Ansteuerung mehrerer Hardwaremodule.

TFT_eSPI Bibliothek

Diese Bibliothek steuert das TFT-Display und ermöglicht die Darstellung von Texten, Grafiken und die Verarbeitung der Touch-Eingaben. Sie bietet einfache Funktionen, um Inhalte auf dem Display zu zeichnen und Touchereignisse auszuwerten

XPT2046_Touchscreen Bibliothek

Diese Bibliothek unterstützt die Touchfunktion des Displays. Sie liest Berührungen aus und liefert die Koordinaten, die das Programm zur Auswahl von Antworten oder Navigation benötigt.

SD.h Bibliothek

Die SD-Bibliothek ermöglicht den Zugriff auf die SD-Karte. Sie erlaubt das Einlesen der Fragen und Antworten, das Anlegen von Dateien sowie das Lesen und Schreiben von Daten.

ArduinoJson Bibliothek

Die ArduinoJson-Bibliothek dient zur Verarbeitung von JSON-Daten. Sie ermöglicht das strukturierte Einlesen der Quizfragen von der SD-Karte, das Auslesen einzelner Felder (Frage, Antworten, richtige Antwort) und deren Verwendung im Programm. Damit kann das Quiz flexibel gestaltet und leicht erweitert werden.

3.3 Zusammenfassung

Die vorgestellten Bauteile und Bibliotheken bilden die Grundlage für das Quizprojekt. Der ESP32-S3 übernimmt die Steuerung, das TFT-Display mit Touch ermöglicht die Interaktion, die SD-Karte speichert die Fragen, und die Bibliotheken unterstützen bei der Programmierung der Anzeige, Touchabfrage und Datenverwaltung. Mit diesen Grundlagen kann das Projekt effizient und flexibel umgesetzt werden.

4 Realisierung

In diesem Kapitel wird die praktische Umsetzung des Projekts beschrieben. Der Fokus liegt auf dem Hardwareaufbau sowie auf der Entwicklung und Durchführung von Testprogrammen zur Überprüfung der einzelnen Komponenten.

Anschließend wird die vollständige Realisierung des Quizsystems dargestellt, einschließlich der Benutzeroberfläche, der Zustandssteuerung sowie der Datenverarbeitung über die SD-Karte.

Die Stromversorgung des Systems erfolgt derzeit über eine USB-Verbindung zum Laptop. Für den späteren Einsatz ist ein kabelloser Betrieb vorgesehen, bei dem das System über eine Powerbank mit Energie versorgt werden soll.

4.1 Hardware-Realisierung

Zunächst wurden alle benötigten Hardware-Komponenten miteinander verbunden. Dazu gehören der ESP32-S3, ein 3,5-Zoll-TFT-Display auf Basis des ILI9488 TFT Display mit integrierter Touchfunktion sowie eine SD-Karte zur Datenspeicherung. Die Verbindungen zwischen den einzelnen Komponenten wurden mithilfe von Jumperkabeln hergestellt.

4.1.1 Verdrahtungsplan des ESP32-S3 mit TFT Display und SD-Karte

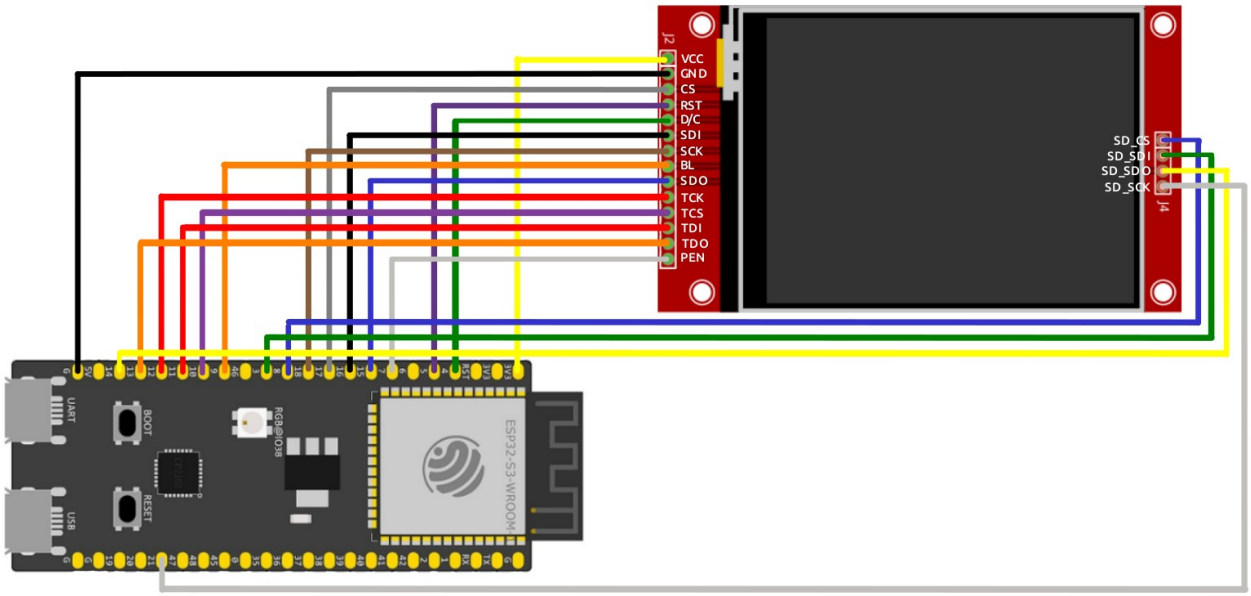


Schaubild 1: Verdrahtungsplan

Der Verdrahtungsplan zeigt die Verbindung zwischen dem ESP32-S3, dem TFT-Display mit integriertem Touchcontroller sowie der SD-Karte. Die einzelnen Komponenten sind über die SPI-Schnittstelle miteinander verbunden, wobei der ESP32-S3 als zentrale Steuereinheit fungiert.

Die Kommunikation erfolgt über die Leitungen MOSI, MISO und SCLK, während für jede Komponente eine separate Chip-Select-Leitung verwendet wird.

4.1.2 Pinbelegungstabelle

Komponente	**Signal**	**ESP32-S3 Pin**
TFT Display	VCC	3.3V
TFT Display	GND	GND
TFT Display	TFT_CS	GPIO 17

TFT Display	TFT_RST	GPIO 5
TFT Display	TFT_DC	GPIO 4
TFT Display	TFT_MOSI	GPIO 16
TFT Display	TFT_SCLK	GPIO 18
TFT Display	TFT_BL	GPIO 9
TFT Display	TFT_MISO	GPIO 15
Touch	TOUCH_SCLK	GPIO 12
Touch	TOUCH_CS	GPIO 10
Touch	TOUCH_MOSI	GPIO 11
Touch	TOUCH_MISO	GPIO 13
Touch	TOUCH_IRQ	GPIO 7
SD-Karte	SD_CS	GPIO 8
SD-Karte	SD_MOSI	GPIO 3
SD-Karte	SD_MISO	GPIO 14
SD-Karte	SD_SCLK	GPIO 21

Bei der Auswahl der GPIO-Pins wurde darauf geachtet, dass die verwendeten Leitungen die jeweiligen Hardwarefunktionen des ESP32-S3 unterstützen. Zudem wurden geeignete Pins für die SPI-Kommunikation (MOSI, MISO, SCLK) gewählt, um eine stabile Datenübertragung zwischen ESP32-S3, Display, Touch und SD-Karte zu gewährleisten.

Für den Touchscreen wurden ebenfalls passende GPIO-Pins verwendet, die eine zuverlässige Kommunikation ermöglichen. Obwohl der ESP32-S3 eigene Touch-Sensor-Pins besitzt, erfolgt die Ansteuerung des verwendeten Touchcontrollers (XPT2046) über die SPI-Schnittstelle, wichtig ist dass diese Touch-Sensor-pins für den Touchcontroller verwendet werden.

Zusätzlich wurden separate Chip-Select-Leitungen für die einzelnen Komponenten definiert, um Konflikte bei der gleichzeitigen Nutzung mehrerer SPI-Geräte zu vermeiden.

4.2 Realisierung der Testfunktionen

Nach dem Hardwareaufbau wurden schrittweise **Testprogramme** entwickelt, um die Funktion der einzelnen Komponenten zu überprüfen.

Zunächst wurde ein Testprogramm für das **TFT-Display und die Touchfunktion** erstellt. Dabei wurde geprüft, ob Inhalte korrekt auf dem Display angezeigt werden und ob Berührungen auf dem Touchscreen zuverlässig erkannt werden. Dieser Test verlief erfolgreich.

Auch wurde die **SD-Karte** angeschlossen und ebenfalls mit einem separaten Testprogramm überprüft. Dabei wurde getestet, ob die SD-Karte korrekt initialisiert wird

und daraufhin eine Testdatei in der SD-Karte erstellt. Auch dieser Test war erfolgreich, sodass sichergestellt ist, dass die SD-Karte funktionsfähig ist und später für das Einlesen der Fragen im JSON-Format verwendet werden kann.

Daraufhin wurde zusätzlich die **ArduinoJson-Bibliothek** getestet, um zu überprüfen, ob die auf der SD-Karte gespeicherte JSON-Datei korrekt ausgelesen und die Inhalte im Programm verwendet werden können. Hierfür wurde ein Testprogramm geschrieben, das die JSON-Datei einliest und die enthaltenen Daten über den **seriellen Monitor** ausgibt. Dieser Test zeigte, dass die Daten korrekt eingelesen und verarbeitet werden können, was eine zuverlässige Grundlage für die spätere Quizlogik darstellt.

Durch diese Tests konnte bestätigt werden, dass die wichtigsten Hardware- und Softwarekomponenten korrekt arbeiten.

4.3 Realisierung der Gesamtfunktion

Nach erfolgreichem Test der einzelnen Funktionen wurden diese zu einem vollständigen Quizsystem zusammengeführt. Beim Start des Programms wird eine Frage aus der JSON-Datei auf der SD-Karte geladen und auf dem Display angezeigt. Die möglichen Antworten werden grafisch dargestellt und können über den Touchscreen ausgewählt werden.

Das Programm wertet die ausgewählte Antwort aus, vergleicht sie mit der richtigen Antwort aus der JSON-Datei und passt den Punktestand entsprechend an.

Der Code wurde schrittweise erweitert. Zunächst wurde die Display-Ausgabe in die bestehenden Zustände als Funktionen integriert. Anschließend wurde die Spiellogik angepasst und um die Touch-Funktionalität ergänzt. Danach folgte die Implementierung der Datenverarbeitung, bei der die Antworten ausgewertet und der Punktestand aktualisiert werden. Zusätzlich wurde eine Funktion zur Darstellung von Zeilenumbrüchen implementiert, um die Textausgabe auf dem Display zu verbessern. Abschließend wurde das System vollständig zusammengeführt und optimiert.

Anschließend wird die nächste Frage geladen .Dieser Ablauf wiederholt sich, bis das Quiz beendet ist bzw bis alle Fragen beantwortet wurden.

Um die Wiederholbarkeit des Spiels zu erhöhen, werden die Fragen vor Beginn des Quiz zufällig gemischt. Dadurch wird sichergestellt, dass die Reihenfolge der Fragen bei jedem Durchlauf unterschiedlich ist.

```
//===============FRAGEN SCHUFFLEN===============
void mischeFragen() {
  for (int i = anzahlFragen - 1; i > 0; i--) {
    int j = random(i + 1);
    int tmp = frageIndex[i];
    frageIndex[i] = frageIndex[j];
    frageIndex[j] = tmp;
  }
}
```

Codeabschnitt 1: Fisher-Yates-Shuffle

Dabei wird ein Array verwendet, das die Reihenfolge der Fragen speichert. Durch zufälliges Vertauschen der Elemente wird eine neue Reihenfolge erzeugt.

4.4 Benutzeroberfläche und Spielablauf

Neben der technischen Umsetzung wurde auch eine grafische Benutzeroberfläche entwickelt, die den Ablauf des Quizspiels darstellt. Die einzelnen Bildschirmansichten wurden mit dem Tool Lopaka erstellt, einem grafischen Editor zur Erstellung von Benutzeroberflächen für Mikrocontroller-Displays.

Für die Umsetzung der Bedienelemente wurde bewusst keine integrierte Button-Klasse der verwendeten Grafikbibliothek (TFT_eSPI_Button) eingesetzt. Stattdessen wurden die Buttons manuell mit grafischen Funktionen wie `drawRect();` und `fillRect()` realisiert, wodurch visuell erkennbare Interaktionsflächen entstehen.

Der Verzicht auf die vorgefertigte Button-Klasse ermöglicht eine höhere Flexibilität bei der Gestaltung und Integration der Benutzeroberfläche, insbesondere im Zusammenspiel mit dem extern generierten Layout aus Lopaka. Zudem bleibt die Implementierung dadurch übersichtlich und unabhängig von zusätzlichen Klassenstrukturen, was die Anpassung und Erweiterung des Interfaces vereinfacht.

Die Touch-Eingaben werden über die abgefragten Koordinaten mit definierten Bildschirmbereichen verglichen, sodass erkannt werden kann, ob ein Benutzer einen bestimmten Bereich berührt hat. Dieses Prinzip wird in allen Bildschirmansichten des Spiels angewendet und ermöglicht eine präzise Steuerung der Benutzerinteraktion.

Die einzelnen Bildschirmansichten sind dabei eng mit dem Zustandsautomaten des Programms verknüpft (siehe Kapitel 4.5), wodurch der Ablauf des Spiels gesteuert wird.

4.4.1 Menübildschirm

Gestaltung 1: Menübildschirm

Der Menübildschirm ermöglicht dem Benutzer den Einstieg in das Quizspiel. Durch Berühren des dargestellten Startbereichs kann das Spiel gestartet werden.

Der oben gezeigte Code-Ausschnitt überprüft, ob sich das System im Zustand `STATE_MENU` befindet (siehe Kapitel 4.5). Anschließend werden die Touch-Koordinaten (x und y) mit einem definierten Bildschirmbereich verglichen, der die Position des dargestellten Buttons beschreibt.

Befindet sich die Berührung innerhalb dieses Bereichs, wird der Zustand des Spiels auf `STATE_START` gesetzt. Dadurch wird der Übergang vom Menü zum eigentlichen Spiel gestartet.

```
// ================= MENU =====================
if (quizState == STATE_MENU) {
  if (x > 90 && x < 390 && y > 120 && y < 180) {
    quizState = STATE_START;
  }
}
```

Codeabschnitt 2: Touchabfrage einer klickfläche im Menu

4.4.2 Spielbeginn-Bildschirm

Gestaltung 2: Startbildschirm

Der Spielbeginn-Bildschirm wird nach dem Start des Quizspiels angezeigt und dient als kurzer Übergang zwischen Menü und erster Frage. In diesem Bildschirm erfolgt keine Benutzereingabe über den Touchscreen. Stattdessen wird eine Meldung angezeigt, die den Beginn des Spiels signalisiert.

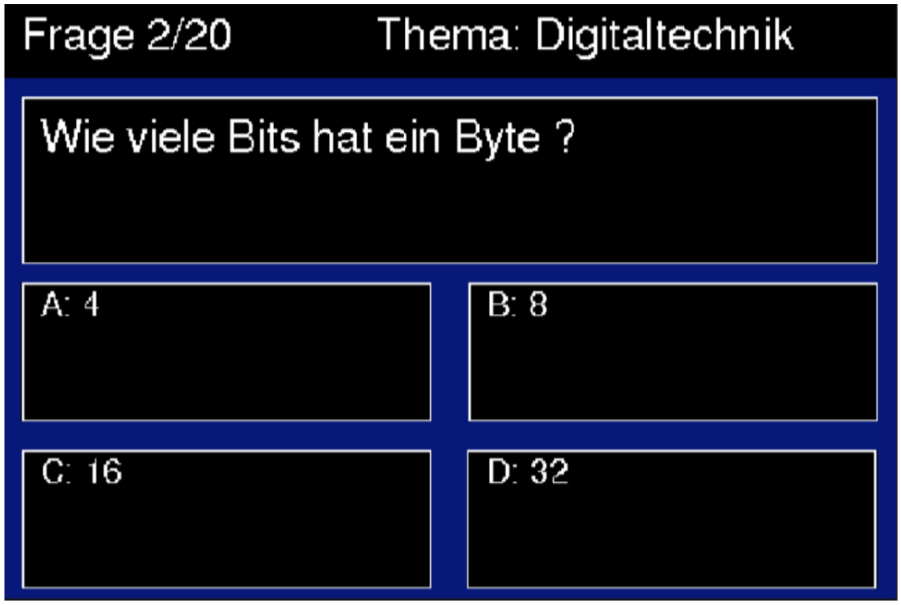
Der Bildschirm bleibt für eine festgelegte Zeit sichtbar und wechselt anschließend automatisch in den nächsten Zustand, in dem die erste Frage angezeigt wird (siehe Kapitel 4.5). Dieser automatische Übergang sorgt für einen flüssigen und benutzerfreundlichen Ablauf des Spiels.

Hier der relevante Code-Ausschnitt dazu:

```
delay(1500);
quizState = STATE_QUESTION;
```

Codeabschnitt 3:
Zustandswechsel nach 1,5 Sekunden

4.4.3 Frage- und Antwortbildschirm



Gestaltung 3: Frage und Antwortbildschirm

Der Frage- und Antwortbildschirm stellt den zentralen Bestandteil des Quizspiels dar. In diesem Bildschirm werden die aktuelle Frage sowie die dazugehörigen Antwortmöglichkeiten angezeigt. Er entspricht dem Zustand `STATE_QUESTION` im Zustandsautomaten (siehe Kapitel 4.5).

Die dargestellten Inhalte werden dynamisch aus einer JSON-Datei auf der SD-Karte geladen (siehe Kapitel 4.6). Die ausgelesenen Daten werden in Variablen gespeichert und anschließend auf dem Display dargestellt.

Der Benutzer kann eine Antwort durch Berühren des entsprechenden Bereichs auf dem Touchscreen auswählen. Die Touch-Koordinaten werden dabei mit definierten Bildschirmbereichen verglichen, die den einzelnen Antwortmöglichkeiten zugeordnet sind.

Sobald eine Berührung erkannt wird, wechselt das System vom Zustand `STATE_QUESTION` in den Zustand `STATE_ANSWER_SELECT`. In diesem Zustand erfolgt die Auswertung der Touch-Eingabe.

Je nach Position der Berührung wird eine der vier Antwortmöglichkeiten ausgewählt. Die gewählte Antwort wird in der Variable `gewaehlteAntwort` gespeichert und visuell hervorgehoben. Anschließend wechselt das System nach einer kurzen Verzögerung in den Zustand `STATE_ANSWER_RESULT`, in dem die Antwort überprüft wird.

Durch diese Struktur wird eine klare Trennung zwischen Anzeige, Auswahl und Auswertung erreicht, was zu einem übersichtlichen und gut erweiterbaren Programmaufbau führt.

Der folgende Code-Ausschnitt zeigt die Auswertung der Touch-Eingabe sowie den Zustandswechsel:

```
// ============== QUESTION ============================
if (quizState == STATE_QUESTION) {
  quizState = STATE_ANSWER_SELECT;
}

  // -------------- ANSWER  SELECT -------------------
if (quizState == STATE_ANSWER_SELECT) {
  int antwort = -1;

  if (y > 150 && y < 225) {
    if (x > 10 && x < 230) antwort = 0;       // A
    else if (x > 250 && x < 470) antwort = 1; // B
  }
  else if (y > 240 && y < 315) {
    if (x > 10 && x < 230) antwort = 2;       // C
    else if (x > 250 && x < 470) antwort = 3; // D
  }
```

Codeabschnitt 4: Antwortauswahl per Koordinaten (A-D)

```
  if (antwort != -1) {
    gewaehlteAntwort = antwort;
    drawAntwortBox(antwort, COLOR_YELLOW);
    delay(1000);
    quizState = STATE_ANSWER_RESULT;
  }
}
```

Codeabschnitt 5: Verarbeitung der Antwortauswahl

Nach der Auswahl einer Antwort erhält der Benutzer eine visuelle Rückmeldung. Zunächst wird die gewählte Antwort hervorgehoben, anschließend wird das Ergebnis farblich dargestellt.

Zur Umsetzung dieser Darstellung wurden eigene Funktionen entwickelt. Diese ermöglichen sowohl den automatischen Zeilenumbruch längerer Texte als auch die kombinierte Darstellung von Text und farbigen Antwortfeldern.

4.4.4 Game-Over-Bildschirm

Gestaltung 4: Gameover-Bildschirm

Der Game-Over-Bildschirm wird angezeigt, sobald alle Fragen des Quizspiels beantwortet wurden. Er stellt den Abschluss des Spiels dar und informiert den Benutzer über das Ende des Quiz.

In diesem Bildschirm werden das erzielte Ergebnis sowie statistische Informationen angezeigt, beispielsweise der erreichte Punktestand und die Anzahl der korrekt beantworteten Fragen. Der Bildschirm entspricht dem Zustand `STATE_GAMEOVER` im Zustandsautomaten (siehe Kapitel 4.5).

Zusätzlich wird dem Benutzer die Möglichkeit gegeben, über einen Button zum Menü zurückzukehren. Durch Berühren des entsprechenden Bereichs auf dem Touchscreen wird der Zustand des Programms wieder auf den Menüzustand gesetzt.

Der Game-Over-Bildschirm sorgt somit für einen klaren Abschluss des Spiels und ermöglicht gleichzeitig einen einfachen Neustart.

Der folgende Code-Ausschnitt zeigt die Auswertung der Touch-Eingabe im Game-Over-Bildschirm sowie den Rücksprung in das Menü:

```
  // ================= GAME OVER =================
  if (quizState == STATE_GAMEOVER) {
    if (x > 90 && x < 390 && y > 250 && y < 300) {
      aktuelleFrage = 0;
      richtigeAnzahl = 0;
      mischeFragen();
      quizState = STATE_MENU;
    }
  }
}
```

Codeabschnitt 6: Rückkehr zum Menü

Dabei werden die Touch-Koordinaten mit dem Bereich des dargestellten Buttons verglichen. Wird dieser Bereich berührt, werden die Spielvariablen zurückgesetzt und die Fragen neu gemischt. Anschließend wird der Zustand des Programmes auf STATE_MENU gesetzt, wodurch der Benutzer wieder zum Menübildschrim gelangt.

4.5 Zustandsautomat

Zur Steuerung des Spielablaufs wird im Programm ein Zustandsautomat verwendet. Dabei wird der aktuelle Spielzustand in der Variable `quizState` gespeichert.

Zunächst werden die möglichen Zustände des Spiels mithilfe eines `enum` definiert:

```
// ================= STATES ===================
enum QuizState {
  STATE_MENU,           // Hauptmenü
  STATE_START,          // "Spiel beginnt"
  STATE_QUESTION,       // Frage anzeigen
  STATE_ANSWER_SELECT,  // Nutzer tippt Antwort
  STATE_ANSWER_RESULT,  // Grün / Rot anzeigen
  STATE_GAMEOVER        // Ende
};
```

Codeabschnitt 7: Deklaration der Zustandsmaschine

Jeder dieser Zustände repräsentiert eine bestimmte Phase des Spiels. Dadurch wird der Ablauf klar strukturiert und übersichtlich dargestellt.

Anschließend werden die Variablen für den aktuellen und den vorherigen Zustand definiert:

```
//=============== Auktueller und voheriger Zustand
QuizState quizState = STATE_MENU;
QuizState lastState = STATE_GAMEOVER;
```

Codeabschnitt 8: Deklarration der Startzustände

Zu Beginn wird der Zustand auf `STATE_MENU` gesetzt, sodass das Spiel im Menü startet. Der Wert von `lastState` wird bewusst unterschiedlich gewählt, damit beim ersten Durchlauf der Hauptschleife der entsprechende Bildschirm sofort gezeichnet wird.

Die eigentliche Steuerung des Zustandsautomaten erfolgt in der `loop()` Funktion:

```
void loop() {
  if (quizState != lastState) {
    lastState = quizState;

    switch (quizState) {
      case STATE_MENU:          drawMenu(); break;
      case STATE_START:         drawStart(); break;
      case STATE_QUESTION:      drawQuestion(); break;
      case STATE_ANSWER_SELECT: break; // nur Touch
      case STATE_ANSWER_RESULT: drawResult(); break;
      case STATE_GAMEOVER:      drawGameOver(); break;
    }
  }

  handleTouch();
}
```

Codeabschnitt 9: Hauptschleife mit Zustandsüberprüfung

Zunächst wird überprüft, ob sich der aktuelle Zustand geändert hat. Ist dies der Fall, wird `lastState` aktualisiert und abhängig vom aktuellen Zustand die entsprechende Funktion aufgerufen. Dadurch wird sichergestellt, dass jeder Bildschirm nur einmal neu gezeichnet wird.

Der Zustand `STATE_ANSWER_SELECT` stellt dabei einen Sonderfall dar, da hier kein neuer Bildschirm angezeigt wird. In diesem Zustand erfolgt ausschließlich die Auswertung der Touch-Eingaben.

Die Funktion `handleTouch()` wird unabhängig vom aktuellen Zustand in jedem Durchlauf der Schleife ausgeführt. Dadurch können Tocuh-eingaben jederzeit erkannt und verarbeitet werden.

Durch die Verwendung dieses Zustandsautomaten wird eine klare Trennung der einzelnen Spielphasen erreicht. Dies verbessert die Übersichtlichkeit des Programms und erleichtert sowohl die Fehlersuche als auch zukünftige Erweiterungen.

4.6 Datenverarbeitung mit Sd-Karte und JSON

Die Quizfragen werden in einer JSON-Datei auf einer SD-Karte gespeichert. Dadurch können die Inhalte des Quiz flexibel angepasst werden, ohne Änderungen am Programmcode vornehmen zu müssen.

Damit die SD-Karte vom ESP32 korrekt erkannt und verwendet werden kann, musste sie im Dateisystem FAT32 formatiert werden.

Beim Start des Programms wird die SD-Karte initialisiert und die entsprechende Datei geöffnet. Hierfür wurde eine eigene Initialisierungsfunktion implementiert, die die

Kommunikation über die SPI-Schnittstelle einrichtet und überprüft, ob die SD-Karte korrekt erkannt wird.

Der folgende Code-Ausschnitt zeigt die grundlegende Initialisierung der SD-Karte:

```
sdSPI.begin(SD_SCLK, SD_MISO, SD_MOSI, SD_CS);

if (!SD.begin(SD_CS, sdSPI)) {
  Serial.println("SD-Karte FEHLER");
  while (1);
}
```

Codeabschnitt 10: SD-Initialisierung

Zunächst werden die verwendeten Pins für die SPI-Kommunikation festgelegt. Anschließend wird geprüft, ob die SD-Karte erfolgreich initialisiert werden konnte.

Nach erfolgreicher Initialisierung wird die JSON-Datei eingelesen und mit der ArduinoJson-Bibliothek verarbeitet. Die Datei enthält alle notwendigen Informationen für das Quiz, darunter die Fragen, Antwortmöglichkeiten und die jeweils richtige Lösung.

Der folgende Ausschnitt zeigt den Aufbau der verwendeten JSON-Struktur:

```
{
  "thema": "Digitaltechnik",
  "beschreibung": "Grundlagen der Digitaltechnik und Logik",
  "fragen": [
    {
      "frage": "Wie viele Bits hat ein Byte?",
      "antworten": ["4", "8", "16", "32"],
      "korrekt": 1
    },
```

Codeabschnitt 11: JSON-Struktur für Fragen und Antworten

In diesem Beispiel enthält das Feld „frage“ die eigentliche Fragestellung. Die Antwortmöglichkeiten sind im Array „antworten“ gespeichert.

Das Feld „korrekt“ gibt die richtige Antwort an und verweist auf den Index der richtigen Lösung innerhalb des Arrays. In diesem Fall entspricht der Wert „1“ der zweiten Antwort („8“).

Während des Spiels werden die benötigten Daten aus der JSON-Datei ausgelesen und in Variablen gespeichert. Diese Variablen werden anschließend verwendet, um die Inhalte auf dem Display darzustellen und die Benutzereingaben auszuwerten.

Durch die Trennung von Daten (JSON-Datei) und Programmcode wird das System übersichtlich und leicht erweiterbar. Neue Fragen oder Themenbereiche können einfach ergänzt werden, ohne Änderungen am Programmcode vornehmen zu müssen.

4.7 Umsetzung der Text- und Antwortdarstellung

Für die Darstellung von Texten wurde ein eigener Font verwendet, da der Standard-Font die Anzeige von Umlauten nicht korrekt unterstützte.

Der Font wurde aus einer TrueType-Schriftdatei (TTF) erzeugt. Hierfür wurde ein Konvertierungswerkzeug der verwendeten Grafikbibliothek genutzt, das die Schrift in ein für das Display geeignetes Format umwandelt.

Die erzeugten Font-Dateien wurden anschließend in das Projekt eingebunden und für die Textdarstellung verwendet. Je nach benötigter Schriftgröße wurden mehrere Varianten des Fonts erzeugt.

Für die Darstellung der Fragen und Antworten wurden mehrere eigene Funktionen entwickelt. Hintergrund war, dass längere Texte auf dem Display nicht automatisch passend umgebrochen werden und zusätzlich eine farbliche Hervorhebung der Antwortfelder benötigt wird.

Zunächst wurde die Funktion `drawStringWrapped()` erstellt. Diese teilt längere Texte, die über dem Buttonrand bzw. Bilschirm laufen in mehrere Zeilen auf. Dabei wird fortlaufend geprüft, wie viele Zeichen noch innerhalb der maximalen Breite dargestellt werden können. Überschreitet der Text die vorgegebene Breite, wird die Zeile abgeschnitten und bei Bedarf mit einem Bindestrich abgeschlossen. Auf diese Weise können auch längere Antworttexte lesbar auf dem Display dargestellt werden.

Der folgende Code-Ausschnitt zeigt das Grundprinzip des Zeilenumbruchs:

```
while (cut <= len) {
  String substr = text.substring(start, cut);
  int w = tft.textWidth(substr);
  if (w > maxWidth) {
    cut--;
    needsHyphen = (cut < len);
    break;
  }
  cut++;
}
String line = text.substring(start, cut);
if (needsHyphen) {
  tft.drawString(line + "-", x, y);
} else {
  tft.drawString(line, x, y);
}
```

Codeabschnitt 12: Berechnung des Zeilenumbruchs mit Silbentrennung

Für die Darstellung der Antworttexte wurde anschließend die Funktion drawAntwortText() verwendet. Diese liest den passenden Antworttext aus der JSON-Datei und gibt ihn an der

vorgesehenen Position aus. Dabei wird die zuvor entwickelte Funktion für den Zeilenumbruch wiederverwendet.

```
JsonObject q = doc["fragen"][frageIndex[aktuelleFrage]];
JsonArray a = q["antworten"];
```

Codeabschnitt 13: Auslesen von Fragen und Antworten

```
tft.setTextColor(TFT_BLACK);   // schwarzer Text auf gelb/grün/rot
tft.setTextSize(1);
tft.setFreeFont(&Arial10pt8b);
drawStringWrapped(String(label) + a[idx].as<const char*>(),x, y,195,20);
```

Codeabschnitt 14: Ausgabe des Antworttextes mit Zeilenumbruch

Darauf aufbauend wurde die Funktion drawAntwortBox() erstellt. Diese zeichnet zunächst das farbige Antwortfeld und gibt anschließend den passenden Antworttext innerhalb des Feldes aus. Dadurch kann dieselbe Funktion sowohl für die normale Anzeige als auch für die farbliche Hervorhebung bei der Auswertung verwendet werden.

```
tft.fillRect(x, y, 220, 75, color);
tft.drawRect(x, y, 220, 75, TFT_WHITE);
drawAntwortText(idx);
```

Codeabschnitt 15: Zeichnen der Antwortboxen mit Rahmen und Textinhalt

Die Funktion drawResult() nutzt die zuvor entwickelten Funktionen, um die Antwortauswertung visuell darzustellen. Dabei werden die entsprechenden Antwortfelder abhängig vom Ergebnis farblich markiert.

Der folgende Code-Ausschnitt zeigt, wie die gewählte Antwort mit der korrekten Lösung verglichen und anschließend farblich dargestellt wird:

```
if (gewaehlteAntwort == richtigeAntwort) {
  drawAntwortBox(gewaehlteAntwort, COLOR_GREEN);
  richtigeAnzahl++;
} else {
  drawAntwortBox(gewaehlteAntwort, COLOR_RED);
  drawAntwortBox(richtigeAntwort, COLOR_GREEN);
}
```

Codeabschnitt 16: Überprüfung der Antwort und farbliche Rückmeldung

Ist die Antwort richtig, wird das entsprechende Antwortfeld grün dargestellt und der Punktestand erhöht. Bei einer falschen Auswahl wird die gewählte Antwort rot markiert, während gleichzeitig die richtige Antwort grün hervorgehoben wird.

5 Fehlersuche, Beisetungen und Verbesserung

Während der Umsetzung des Projekts traten verschiedene Probleme und Fehlversuche auf, die im Rahmen der Fehlersuche analysiert und schrittweise behoben wurden.

Die Lösungsansätze basierten sowohl auf eigener Analyse als auch auf der Nutzung externer Informationsquellen sowie unterstützender Hinweise der Lehrkraft.

Dieses Kapitel beschreibt die aufgetretenen Probleme, deren Ursachen sowie die getroffenen Maßnahmen und daraus gewonnenen Erkenntnisse.

5.1 Fehlersuche und Fehlerursachen

Während der Umsetzung des Projekts traten verschiedene Probleme und Fehlversuche auf, die die Funktion einzelner Komponenten beeinträchtigten.

Die Ursachen lagen sowohl in fehlerhafter Verkabelung als auch in ungeeigneten Softwarekonfigurationen und falschen Einstellungen einzelner Komponenten.

Die folgenden Abschnitte beschreiben die wichtigsten aufgetretenen Probleme sowie deren jeweilige Ursachen.

5.1.1 Fehler bei der SPI-Verkabelung des TFT-Displays

Zu Beginn der Projektumsetzung trat ein schwerwiegendes Problem auf: Der **ESP32-S3 startete wiederholt neu (Reboot)**, während das TFT-Display lediglich **weiß blieb** und keine Inhalte anzeigte. Ein normaler Programmablauf war dadurch nicht möglich.

Nach Rücksprache mit dem Lehrer wurde ein angepasstes Beispielprogramm mit **korrekten Pin-Definitionen** zur Verfügung gestellt. Nach dem Einspielen dieses Codes reduzierte sich das Problem der ständigen Reboots, jedoch blieb das Display weiterhin weiß.

Die Ursache des Problems lag schließlich in einer **fehlerhaften Verkabelung der SPI-Leitungen**. Die Pins **MOSI und MISO waren vertauscht**, wodurch keine korrekte Datenübertragung zwischen ESP32-S3 und TFT-Display stattfinden konnte.

5.1.2 Fehler bei der SD-Karten-Verkabelung

Bei der Integration der SD-Karte traten ebenfalls Probleme auf. Zunächst wurde versucht, das TFT-Display und die SD-Karte über **gemeinsam genutzte Pins** zu betreiben. Dazu wurden zwei Jumperkabel zu einem sogenannten **Dual-Jumperkabel** zusammengelötet, um die SPI-Leitungen zu teilen.

Diese Vorgehensweise führte jedoch zu **Störungen in der Kommunikation**, wodurch die **Initialisierung der SD-Karte nicht erfolgreich war**. Auch nach der Verwendung einer korrekt formatierten SD-Karte zeigte das Programm weiterhin die Meldung **„SD fail“**.

Die Ursache dieses Problems lag erneut in einer **fehlerhaften Verkabelung**. Die Leitungen **MOSI und MISO der SD-Karte waren vertauscht**, wodurch keine korrekte Kommunikation zwischen ESP32-S3 und SD-Karte stattfinden konnte.

5.1.3 SPI-Bus- und Ressourcen-Konflikt beim ESP32-S3

Zu Beginn des Projekts wurde das System **ohne SD-Karte** umgesetzt. In dieser Phase wurden lediglich das TFT-Display und der Touchscreen verwendet. Die grundlegende Funktion des Systems war dabei gegeben, sodass erste Tests erfolgreich durchgeführt werden konnten.

Im weiteren Projektverlauf wurde das System um eine **SD-Karte** erweitert. Ab diesem Zeitpunkt traten Probleme auf, insbesondere bei der Touchfunktion. Der Touchscreen reagierte nicht mehr zuverlässig, obwohl die Pin-Nummern im Code korrekt definiert waren.

Ursprünglich wurde der Touchscreen über den SPI-Bus VSPI eingebunden, während für die SD-Karte ein anderer SPI-Bus verwendet wurde. Diese Bezeichnungen sind von älteren ESP32-Modellen bekannt, bei denen VSPI und HSPI als getrennte Hardware-Schnittstellen genutzt werden können.

Beim ESP32-S3 existiert diese Struktur jedoch nicht in gleicher Weise. Stattdessen wird unter anderem der SPI-Controller FSPI verwendet. Die Verwendung des VSPI-Busses führte daher zu einer fehlerhaften oder nicht funktionierenden Kommunikation mit dem Touchscreen.

Zusätzlich kam es zu einem Ressourcenkonflikt, da sowohl der Touchscreen als auch die SD-Karte auf denselben SPI-Controller zugriffen. Dadurch konnten nicht beide Komponenten gleichzeitig korrekt betrieben werden.

Darüber hinaus wurde die SPI-Schnittstelle zunächst nicht vollständig initialisiert, wodurch es zu einer fehlerhaften Zuordnung der Pins kam.

5.2 Fehlerbeseitungen

5.2.1 Fehlerbehebung – TFT- Display(Reboot und weißer Bildschirm)

Nach dem Erkennen er fehlerhaften Verkabelung wurden die **MOSI- und MISO-Leitungen des TFT-Displays korrekt angeschlossen**. Anschließend wurde das System erneut getestet.

Nach dieser Korrektur startete der ESP32-S3 stabil, und das TFT-Display funktionierte wie vorgesehen.Texte und grafische Inhalte konnten nun zuverlässig angezeigt werden

5.2.2 Fehlerbehebung – SD-Karte (Initialisierung und SD“fail“)

Nach dem Korrigieren der **MOSI- und MISO-Verbindungen der SD-Karte** wurde das System erneut getestet.

Anschließend funktionierte die Initialisierung der SD-Karte zuverlässig, und auf die gespeicherte Testdatei konnte erfolgreich zugegriffen werden.

5.2.3 Fehlerbeseitung zum SPI-Bus und Ressourcen Konflikt beim ESP32S3

Zur Behebung des Problems wurde die SPI-Kommunikation **explizit konfiguriert**. Dabei wurden die verwendeten GPIO-Pins eindeutig über die GPIO-Matrix des ESP32-S3 zugewiesen.

Vorher:

- **tsSPI.begin(TOUCH_SCLK, TOUCH_MISO, TOUCH_MOSI, TOUCH_CS);**

nachher:

- **tsSPI.begin(12, 13, 11, 10);**

Zusätzlich wurden die beiden Peripheriegeräte bewusst auf **zwei unterschiedliche Hardware-SPI-Busse** verteilt:

vorher:

- **VSPI** (SPI3) für den Touchscreen
- **FSPI** (SPI3) für die SD-Karte

nacher:

- **FSPI** (SPI3) für den Touchscreen
- **HSPI** (SPI2) für die SD-Karte

Durch diese klare Trennung der Hardware-Ressourcen und die vollständige Initialisierung der SPI-Busse konnte eine störungsfreie Kommunikation zwischen ESP32-S3, Touchscreen und SD-Karte sichergestellt werden.

5.3 Erkenntnisse und Verbesserungen

Durch die aufgetretenen Probleme wurde deutlich, dass das Teilen von SPI-Leitungen zwar grundsätzlich möglich ist, jedoch eine sorgfältige Planung sowie eine saubere und eindeutige Verdrahtung erfordert.

Für dieses Projekt hat sich die Verwendung getrennter Pins als deutlich zuverlässigere Lösung erwiesen, da dadurch Konflikte zwischen den einzelnen Komponenten vermieden werden konnten.

Für zukünftige Projekte empfiehlt es sich, die Pinbelegung bereits vor dem Aufbau eindeutig zu dokumentieren und alle Verbindungen klar zu kennzeichnen. Auf diese Weise können Verwechslungen, insbesondere zwischen MOSI und MISO, vermieden werden.

Darüber hinaus zeigte sich während der Entwicklung, dass auch die Darstellung von Texten und die Benutzeroberfläche optimiert werden können. Der zunächst verwendete Standard-Font unterstützte die Darstellung von Umlauten nur unzureichend, was zu

fehlerhaften oder unleserlichen Zeichen führte. Durch die Verwendung eines geeigneten Fonts konnte die Lesbarkeit deutlich verbessert werden.

Zusätzlich wurde eine eigene Funktion zur Darstellung von Texten mit Zeilenumbruch entwickelt, da längere Texte nicht automatisch korrekt auf dem Display dargestellt wurden. Diese Anpassung ermöglichte eine übersichtliche und strukturierte Anzeige der Inhalte und verbesserte die Benutzerfreundlichkeit des Systems erheblich.

6 Ergebnisse

Im Rahmen des Projekts wurde ein funktionsfähiges Quizsystem auf Basis des ESP32-S3 erfolgreich umgesetzt. Das System kombiniert ein TFT-Display mit Touchfunktion sowie eine SD-Karte zur Speicherung der Quizfragen.

Das entwickelte System ermöglicht es, Fragen aus einer externen JSON-Datei zu laden und diese auf dem Display darzustellen. Die zugehörigen Antwortmöglichkeiten werden übersichtlich angezeigt und können über den Touchscreen ausgewählt werden.

Die Benutzereingaben werden zuverlässig erkannt und ausgewertet. Je nach Auswahl erhält der Benutzer eine direkte visuelle Rückmeldung, wobei richtige Antworten grün und falsche Antworten rot dargestellt werden. Zusätzlich wird die richtige Lösung hervorgehoben.

Der gesamte Ablauf des Quizspiels wird durch einen Zustandsautomaten gesteuert, wodurch ein strukturierter Übergang zwischen den einzelnen Spielphasen (Menü, Spielstart, Frage, Auswertung und Spielende) gewährleistet ist.

Die Verwendung einer SD-Karte in Kombination mit dem JSON-Format ermöglicht eine flexible Erweiterung des Systems. Neue Fragen können ohne Anpassung des Programmcodes hinzugefügt werden.

Darüber hinaus wurde die Benutzeroberfläche so gestaltet, dass sie übersichtlich und leicht verständlich ist. Durch den Einsatz eigener Funktionen zur Textdarstellung, wie beispielsweise dem Zeilenumbruch und der Verwendung eines geeigneten Fonts, konnte die Lesbarkeit deutlich verbessert werden.

Darüber hinaus wurde die Benutzeroberfläche so gestaltet, dass sie übersichtlich und leicht verständlich ist. Durch den Einsatz eigener Funktionen zur Textdarstellung, wie beispielsweise dem Zeilenumbruch und der Verwendung eines geeigneten Fonts, konnte die Lesbarkeit deutlich verbessert werden.

6.1 Gehäuseentwurf

Zusätzlich zur elektronischen Umsetzung wurde ein Gehäuse für das System entworfen. Dieses wurde mit dem CAD-Programm Tinkercad modelliert.

Als Grundlage diente ein bereits vorhandenes Gehäusemodell aus einer frei zugänglichen Online-Quelle, welches an die Anforderungen des Projekts angepasst wurde. Dabei

wurden insbesondere die Abmessungen des Gehäuses verändert sowie zusätzliche Aussparungen für den SD-Karten-Anschluss und den USB-C-Anschluss integriert.

Ziel war es, die einzelnen Komponenten wie den ESP32-S3 und das TFT-Display in einem gemeinsamen Gehäuse unterzubringen. Dafür wurde ausreichend Platz für die Bauteile sowie deren Anschlüsse vorgesehen.

Die folgenden Abbildungen zeigen das ursprüngliche Modell, die angepasste Version sowie das gedruckte Gehäuse mit eingebauten Komponenten. Dadurch wird der gesamte Entwicklungsprozess von der Vorlage bis zur praktischen Umsetzung nachvollziehbar dargestellt.

Das Gehäuse wurde zunächst digital konstruiert und anschließend durch 3D-Druck umgesetzt.

Die Komponenten wurden im Gehäuse nicht wie ursprünglich vorgesehen verschraubt, sondern mit Heißkleber fixiert.

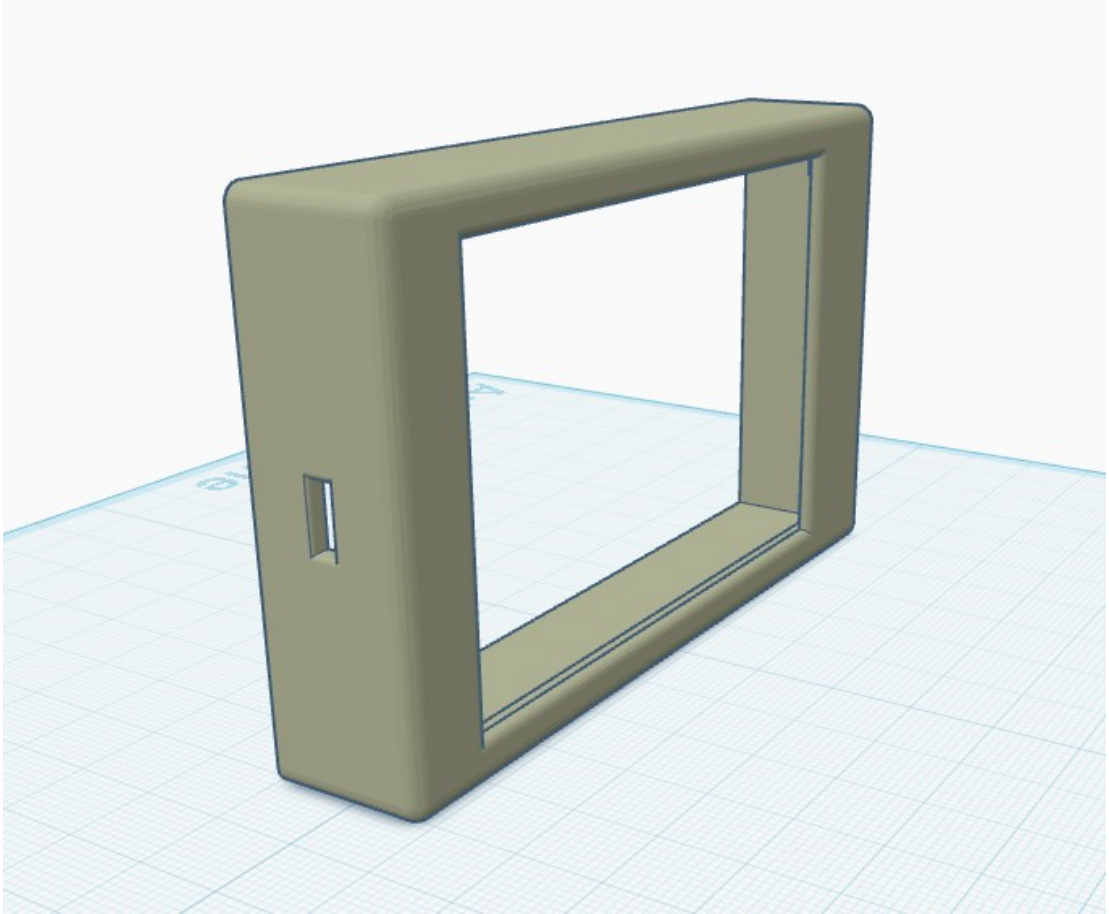


Abbildung 1: Ursprüngliches Gehäusemodell aus einer frei zugänglichen Online-Quelle

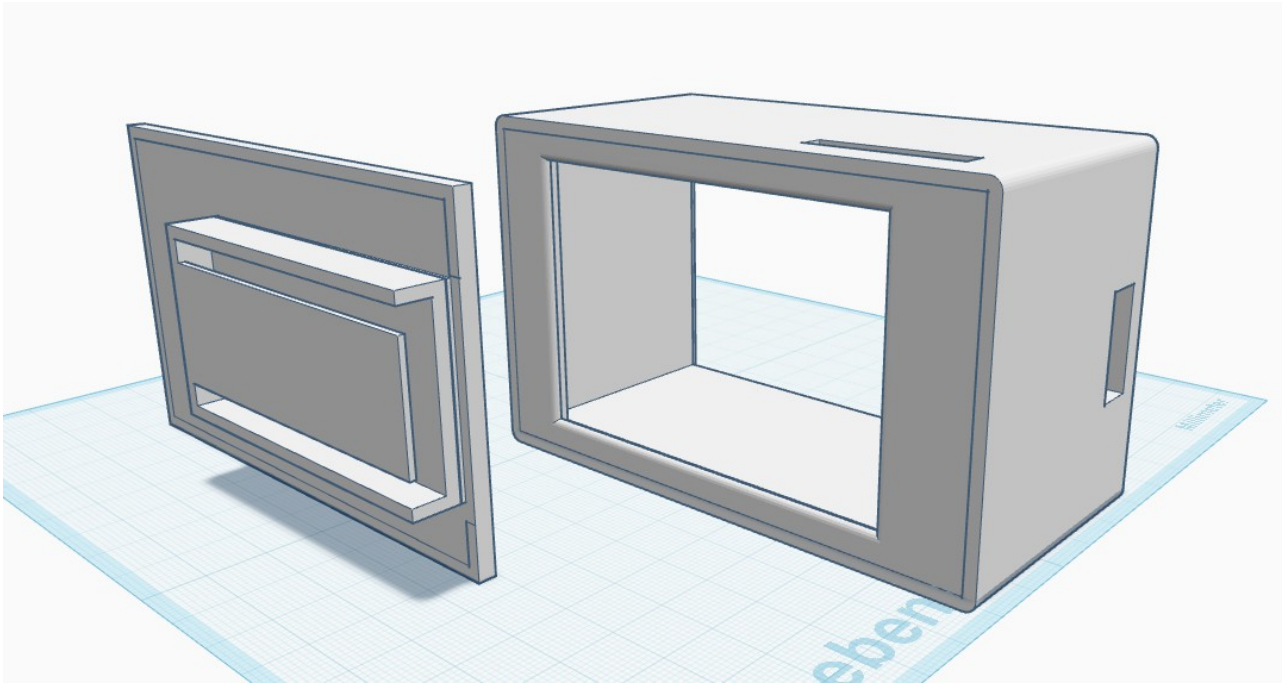


Abbildung 2: Angepasstes Gehäusemodell in Tinkercad mit veränderter Gehäusebreite, integriertem Deckel sowie zusätzlichen Aussparungen für den SD-Karten- und USB-C-Anschluss

Abbildung 3: 3D-gedrucktes Gehäuse mit eingebauten und im Gehäuse befestigten Komponenten

7 Fazit

Im Rahmen dieses Projekts wurde ein interaktives Quizsystem auf Basis des ESP32-S3 erfolgreich entwickelt und umgesetzt. Ziel war es, ein funktionierendes System zu realisieren, das Fragen aus einer externen Datei lädt, auf einem Display darstellt und über einen Touchscreen bedienbar ist.

Dieses Ziel konnte vollständig erreicht werden. Das entwickelte System arbeitet zuverlässig und ermöglicht einen strukturierten Ablauf des Quizspiels. Die Kombination aus Display, Touchfunktion und externer Datenspeicherung stellt eine vielseitige und flexible Lösung dar.

Während der Umsetzung konnten zahlreiche praktische Erfahrungen im Bereich der Mikrocontroller-Programmierung, der Hardware-Verdrahtung sowie der Fehlersuche gesammelt werden. Insbesondere die Arbeit mit der SPI-Schnittstelle und die Integration mehrerer Komponenten stellten eine Herausforderung dar, die erfolgreich bewältigt werden konnte.

Zusätzlich wurden erste Erfahrungen im Bereich der 3D-Konstruktion gesammelt, indem ein Gehäuse für das System mit einem CAD-Programm entworfen wurde.

Darüber hinaus wurden eigene Lösungen entwickelt, um die Benutzeroberfläche zu verbessern. Dazu zählen unter anderem die Verwendung eines geeigneten Fonts zur korrekten Darstellung von Umlauten sowie die Implementierung eines Zeilenumbruchs für längere Texte. Diese Anpassungen führten zu einer deutlich besseren Lesbarkeit und Benutzerfreundlichkeit.

Rückblickend zeigte sich, dass eine sorgfältige Planung der Hardware und eine strukturierte Vorgehensweise bei der Fehlersuche entscheidend für den Projekterfolg sind.

Insgesamt stellt das Projekt eine erfolgreiche Umsetzung der gestellten Aufgabe dar und zeigt die Möglichkeiten eines Mikrocontroller-basierten Systems in Kombination mit grafischer Benutzeroberfläche und externer Datenspeicherung.

8 Anhang und Quellen

8.1 Programmcode

Der vollständige Quellcode sowie alle projektrelevanten Dateien sind in einem GitHub-Repository gespeichert.

https://github.com/T0ni714/Projektarbeit_Antonio

8.2 Nutzung von KI

Im Rahmen der Projektdokumentation wurden KI-gestützte Tools wie ChatGPT , Gemini und Claude unterstützend eingesetzt.

Die Nutzung erfolgte insbesondere bei der Programmierung, der Fehlersuche sowie der strukturellen und sprachlichen Ausarbeitung der Dokumentation. Dabei wurden gezielt Fragen zu einzelnen Codeabschnitten, zur Umsetzung von Funktionen sowie zur Behebung von Fehlern gestellt

8.2.1 Prompt für die Software Struktur

Als Grundlage für die Softwareentwicklung diente ein von der betreuenden Lehrkraft bereitgestellter Test-Sketch zur Ansteuerung des TFT-Displays und des Touchscreens

Benutzte prompts:

- „Wie kann ich auf Basis des Test sketches eine strukturierte Quiz-Software mit Menu, Frage und touch logik bauen. (Test sketch ausschnitt reinkopiert).“
- „Wie kann ich Quizfragen von einer SD-Karte im JSON-Format laden und im Programm verwenden? Was muss ich bei SD formatierung achten ?“

Die KI schlug dabei unter anderem die Verwendung eines Zustandsautomaten sowie das Laden der Fragen über eine JSON-Datei vor. Zusätzlich wurden Vorschläge zur Ergänzung und Verbesserung bestehender Codeabschnitte gegeben.

8.2.2 Prompt für den Software-Ablauf

Benutzte prompts:

- „Implementiere die Logik eines Quizspiels im Stil von Wer wird Millionär.“
- „Wie kann ich Fragen randomizen?“
- „gib mir 20 Fragen aus dem Bereich Digitaltechnick im Json format“

Es wurden Lösungen für einzelne Teilprobleme der Spiellogik erarbeitet, insbesondere für die Umsetzung des Quiz-Ablaufs sowie das zufällige Mischen der Fragen.

Erstellt am:

8.2.3 Prompt-Optimierung für Umlaute und Zeilenumbrüche

Benutzte prompts:

- „online font tools funktionieren nicht ? kann man es im Linux Terminal auch machen und wie ?“
- „muss ich für jede Font größe eine neue .h Datei erstellen ?“

- Mein Text im Display geht überm Rand heißt. es hat keine Zeilenumbruch und überlappt dadurch andere Texte, wie füge ich einen manuellen Zeilenumbruch hinzu mit einem Bindestrich am ende?

Es wurde ein neuer Font eingebunden, der Umlaute (ä, ö, ü) korrekt unterstützt. Ebenfalls wurde eine Lösung zur verbesserten Textdarstellung auf dem Display erarbeitet, sodass lange Texte korrekt umgebrochen werden und keine Überlappungen mehr entstehen. Dabei wird am Zeilenende ein Bindestrich gesetzt.

8.3 Verwendete Bibliotheken

Die im Projekt verwendeten Bibliotheken wurden bereits in Kapitel 2.2 und 3.2 beschrieben. Zur Übersicht sind diese hier nochmals zusammengefasst:

- TFT_eSPI(von Bodmer): Verfügbar über den **Arduino Library Manager**
- ArduinoJson (von Benoît Blanchon): Verfügbar über den **Arduino Library Manager**
- SD.h: Standardbibliothek (Teil des **ESP32-Arduino-Core**, keine separate Installation nötig).
- XPT2046_Touchscreen (von Paul Stoffregen): aus einer externen Quelle (GitHub)

8.4 Web-Quellen

Infos über das Touchscreen:

https://www.lcdwiki.com/3.5inch_SPI_Module_ILI9488_SKU:MSP3520

Display Bauplan:

https://www.lcdwiki.com/res/MSP3520/MSP3520_Size.pdf

weiße Display Problem:

https://forum.arduino.cc/t/xiao-esp32s3-und-tft-espi-mit-ili9488/1254104

TFT_eSPI Grundlagen:

https://doc-tft-espi.readthedocs.io/tft_espi/methods/constructor/

SPI bus Input:

https://curiousscientist.tech/blog/esp32-s3-multiple-spi-buses-spi-fspi-hspi

XPT2046_Touchscreen:

https://github.com/PaulStoffregen/XPT2046_Touchscreen

Gehäuse vorlage:

https://www.thingiverse.com/thing:5397203b

ESP32-S3 N16R8/N8R2

https://de.aliexpress.com/item/1005006266375800.html?spm=a2g0o.order_list.order_list_main.32.703b5c5f8EbxOq&gatewayAdapt=glo2deu

3, 5 ILI9488 TFT touchscreen

https://de.aliexpress.com/item/1005006661261223.html?spm=a2g0o.order_list.order_list_main.10.703b5c5f8EbxOq&gatewayAdapt=glo2deu

Lopaka

https://lopaka.app/projects

thinkercad für die Umsetzung des 3D Modell

https://www.tinkercad.com/dashboard

9 Bedienungsanleitung

Nach dem Einschalten des Systems erscheint zunächst das Hauptmenü. Über den angezeigten Button kann das Quiz gestartet werden.

Nach dem Start wird kurz ein Startbildschirm angezeigt, bevor automatisch die erste Frage erscheint.

Jede Frage wird mit vier Antwortmöglichkeiten dargestellt. Der Benutzer kann eine Antwort durch Berühren des entsprechenden Feldes auswählen.

Nach der Auswahl wird die gewählte Antwort zunächst hervorgehoben. Anschließend wird das Ergebnis angezeigt: Richtige Antworten werden grün markiert, falsche rot. Zusätzlich wird die richtige Lösung angezeigt.

Nach der Auswertung wird automatisch zur nächsten Frage gewechselt.

Nachdem alle Fragen beantwortet wurden, erscheint ein Game-Over-Bildschirm. Dort besteht die Möglichkeit, über einen Button zurück zum Hauptmenü zu gelangen und das Quiz erneut zu starten.

10 English Summary

“Who Wants to Be a Millionaire” Quiz with ESP32- S3 and TFT Display

In this project I developed an interactive quiz about digital technology using an **ESP32-S3 -Microcontroller** and a **3.5-inch-ILI9488 TFT-display**. The idea was inspired by the TV show „Who Wants to Be a Millionaire ?“, but instead of general knowledge questions, the quiz contains questions about digital technology.

The goal of the project was to create a learning tool that helps users test their knowledge of digital technology in a fun and interactive way. At the same time, I wanted to gain experience with microcontrollers, programming, and graphical displays.

For this project, I used an ESP32-S3 microcontroller, which is responsible for the game logic and control of the game, a TFT display that presents the questions and answers and to enter the answers via touch, an SD card to load up the questions and answers needed for the quiz and last but not least, a power bank that provides a portable and wireless power supply for the device.

The quiz works similarly to the „Who Wants to Be a Millionaire“ game. The display shows one question and four possible answers. The questions are related to topics from digital technology, for example logic gates, binary numbers and basic digital circuits.

All questions are stored on an SD card. The ESP32-S3 reads the questions and answer options from the SD card and displays them on the TFT screen. This makes it easy to add or change questions without modifying the whole program which includes roughly 400 lines.

The player can select an answer by touch the screen. After the player chooses an answer, the program checks if it is correct. If the answer is correct, the player earns a point and the next question appears on the screen. If the answer is incorrect, no point is given and the next question is displayed.

To create the project, I programmed the ESP32-S3 using the Arduino IDE. I also designed the graphical interface for the TFT display so that the questions and answers are clearly visible. During development, I tested the system several times to ensure that everything worked correctly.

At the end of the project, I successfully created a working digital technology quiz that runs on the ESP32-S3 with a TFT display. The system allows users to answer questions and test their knowledge in an interactive way.

During this project, I learned more about programming microcontrollers, working with displays, and implementing game logic in embedded systems.

In the future, I could improve my project by adding more features. For example, it would be possible to implement a 50/50 joker similar to the one used in the original „Who Wants to Be a Millionaire“. When the player uses this joker, two wrong answers would be removed from the four answers options, which could make the game more interactive.

However, my main plan is to design and build a 3D-printed case for the hardware .A case would protect the ESP32-S3 and the ILI9488 display and make the project look more professional and finished. It would also make the device easier to handle and present.

Overall, this project helped me combine technical knowledge about digital electronics with practical programming skills.

Antonio Okwarauba

Erklärung

Die vorliegende Projektarbeit habe ich selbständig und nur mit den abgegebenen Hilfsmitteln angefertigt.

Alle Stellen, die dem Sinn oder dem Wortlaut nach anderen Werken entnommen sind, wurden durch Angabe der Quellen kenntlich gemacht. Alle wörtlich entnommenen Stellen sind als Zitate gekennzeichnet.

Mannheim, den _____ Unterschrift_____